



**Кафедра информационных систем
в химической технологии**

**Институт тонких химических технологий
им. М.В. Ломоносова
РТУ МИРЭА**



ПРОГРАММИРОВАНИЕ В ЗАДАЧАХ ХТ

Лекция 13. Пакет SciPy

Содержание лекции

В данной лекции будут рассмотрены следующие темы:

- 🌀 Обзор библиотеки SciPy
- 🌀 Основные модули библиотеки SciPy
 - ◉ Численное интегрирование и решение ОДУ
 - ◉ Оптимизация и нахождение корней
 - ◉ Линейная алгебра
 - ◉ Быстрое преобразование Фурье
 - ◉ Обработка сигналов
 - ◉ Статистика
 - ◉ Интерполяция
 - ◉ Обработка изображений
 - ◉ Работа с пространственными данными
 - ◉ Работа с вводом/выводом

Пакет SciPy

- 🔊 SciPy (Scientific Python) - это мощная библиотека для научных и технических вычислений в Python. Она построена на основе NumPy и предоставляет множество удобных функций для работы с математическими алгоритмами, статистикой, оптимизацией, обработкой сигналов и т.д.
- 🔊 SciPy используют специалисты по Data Science, Big Data, аналитики данных, а также математики и ученые:
 - ⦿ для сложных математических расчетов, которые тяжело произвести вручную или с помощью калькулятора;
 - ⦿ проведения научных исследований, где требуется использование продвинутой математики;
 - ⦿ глубокого анализа данных, интерполяции и других методов работы с информацией;
 - ⦿ машинного обучения и создания моделей искусственного интеллекта, прогнозирования и построения моделей;
 - ⦿ формирования двумерных и трехмерных графиков, которые можно потом визуализировать (уже при помощи других библиотек).

Отличия SciPy от NumPy

Основные различия между библиотеками:

- ⦿ в SciPy гораздо больше функций и методов, чем в NumPy;
- ⦿ NumPy ориентирована на базовые вычисления и простую работу с матрицами, SciPy предназначена для глубокого научного анализа;
- ⦿ NumPy не имеет дополнительных зависимостей, вместе с библиотекой не нужно ничего устанавливать. SciPy требует установки NumPy для корректной работы.

SciPy расширяет возможности NumPy, а некоторые частые действия в ней реализованы как отдельные функции, поэтому библиотека сильно упрощает работу со сложными задачами с использованием продвинутой математики

Основные модули SciPy

Подмодуль	Назначение
<code>scipy.integrate</code>	Численное интегрирование
<code>scipy.optimize</code>	Оптимизация (минимизация/максимизация)
<code>scipy.linalg</code>	Линейная алгебра
<code>scipy.fft</code>	Быстрое преобразование Фурье
<code>scipy.signal</code>	Обработка сигналов
<code>scipy.stats</code>	Статистика
<code>scipy.interpolate</code>	Интерполяция
<code>scipy.spatial</code>	Работа с расстояниями, KD-деревьями (например, поиск ближайших соседей)
<code>scipy.ndimage</code>	Обработка изображений
<code>scipy.io</code>	Чтение/запись данных (MATLAB, CSV)

Интегрирование (модуль `scipy.integrate`)



Возможности:

- ⦿ Численное интегрирование функций (определённый интеграл).
- ⦿ Решение обычных дифференциальных уравнений (ОДУ).

Функция	Назначение
<code>quad</code>	Одномерное интегрирование
<code>dblquad</code> , <code>tplquad</code>	Двойные и тройные интегралы
<code>solve_ivp</code>	Решение ОДУ
<code>odeint</code> (устаревшая)	Старый способ решения ОДУ

Интегрирование (модуль `scipy.integrate`)

Возможности:

- ⦿ Численное интегрирование функций (определённый интеграл).
- ⦿ Решение обычных дифференциальных уравнений (ОДУ).

Примеры:

Интеграл и решение ОДУ ($dy/dx = -2y$)

```
from scipy.integrate import quad

result, _ = quad(lambda x: x**2, 0, 3)
print(result)
```

`quad(func, a, b)`

***func** – функция*

***a** – начало промежутка интегрирования*

***b** – конец промежутка*

Интегрирование (модуль `scipy.integrate`)

```
from scipy.integrate import solve_ivp

def ode(t, y): return -2*y
sol = solve_ivp(ode, [0, 5], [1])
print(sol.y[0][-1]) # Приблизительное значение y в t=5
```

ode (название задали сами) — функция-правило: задаёт дифференциальное уравнение.

[0, 5] — интервал интегрирования (от $t = 0$ до $t = 5$).

[1] — начальное условие:

y(0)=1.

solve_ivp возвращает объект *OdeResult*, содержащий:

t — массив точек времени.

y — массив значений решения (в данном случае $y(t)$ для каждого t).

Оптимизация и нахождение корней (модуль `scipy.optimize`)

Возможности:

- ⦿ Минимизация/максимизация функций.
- ⦿ Решение уравнений (нахождение корней).
- ⦿ Линейное программирование.



Функция	Назначение
<code>minimize</code>	Минимизация скалярной функции
<code>root</code>	Поиск корня (нулевого значения функции)
<code>fsolve</code>	Численное решение нелинейных систем
<code>curve_fit</code>	Нелинейная подгонка данных
<code>linprog</code>	Линейное программирование

Оптимизация и нахождение корней (модуль `scipy.optimize`)

 Примеры:

Минимизация функции $y=(x-3)^2$

```
from scipy.optimize import minimize

f = lambda x: (x - 3)**2
res = minimize(f, x0=0)
print(res.x)  # [3.] y=0, x=3
```

Оптимизация и нахождение корней (модуль `scipy.optimize`)

Примеры:

Решение уравнения $f(x) = x^3 - 1 = 0$

```
from scipy.optimize import root

sol = root(lambda x: x**3 - 1, 0.5)
print(sol.x) # [1.]
```

Функция `root` используется для нахождения корней уравнений или систем уравнений. Работает как численный решатель: она "угадывает" решение и итеративно приближает его.

0.5 – это начальное приближение.

Оно влияет на то, к какому корню сойдётся решатель (актуально, если корней несколько).

`sol` – это результат работы `root`. Он содержит много информации (успешность, количество итераций и др.).

Оптимизация и нахождение корней (модуль `scipy.optimize`)

Примеры:

Решить систему из двух уравнений:

1. $x + y^2 = 4$
2. $e^x + x*y = 3$

```
import numpy as np
from scipy.optimize import fsolve

# Определяем систему уравнений
def equations(vars):
    x, y = vars # распаковываем переменные
    eq1 = x + y**2 - 4 # первое уравнение:  $x + y^2 - 4 = 0$ 
    eq2 = np.exp(x) + x*y - 3 # второе уравнение:  $e^x + x*y - 3 = 0$ 
    return [eq1, eq2] # возвращаем список уравнений

# Начальное предположение
initial_guess = [1, 1]

# Решаем систему
solution = fsolve(equations, initial_guess)

# Выводим результат
print(f"Решение: x = {solution[0]:.4f}, y = {solution[1]:.4f}")
```

Линейная алгебра (модуль `scipy.linalg`)



Возможности:

- ⦿ решения систем уравнений,
- ⦿ нахождения определителей, собственных значений,
- ⦿ сингулярного разложения (SVD) и др.

Функция	Назначение
<code>solve</code>	Решение $Ax = b$
<code>inv</code>	Обратная матрица
<code>eig</code>	Собственные значения и векторы
<code>det</code>	Определитель
<code>svd</code>	Сингулярное разложение

Линейная алгебра (модуль `scipy.linalg`)

Пример:

Собственные значения

```
from scipy.linalg import eig
import numpy as np

A = np.array([[1, 2], [2, 3]])
values, vectors = eig(A)
print(values)
```

Этот код:

Импортирует функцию `eig` из `scipy.linalg` – для нахождения собственных значений и собственных векторов матрицы.

Создаёт матрицу A : $A = \begin{bmatrix} 1 & 2 \\ 2 & 3 \end{bmatrix}$

Вызывает `eig(A)`, чтобы найти:

- собственные значения (возвращаются в `values`),
- собственные векторы (в `vectors`).

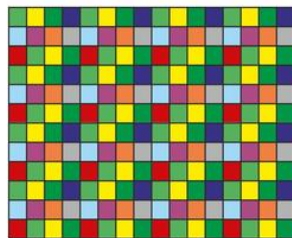
Выводит собственные значения.

Интерполяция (модуль `scipy.interpolate`)

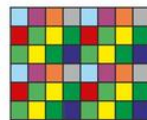
Возможности:

- Предназначен для аппроксимации по известным данным.

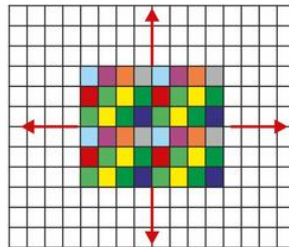
Функция	Назначение
<code>interp1d</code>	Линейная/сплайн-интерполяция 1D
<code>griddata</code>	Интерполяция по сетке
<code>UnivariateSpline</code>	Сплайн-функции
<code>PchipInterpolator</code>	Монотонная интерполяция



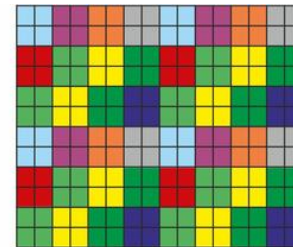
Высокое разрешение
(чёткая картинка,
много точек изображения)



Низкое
разрешение
(немного точек
изображения)



«Размазываем» каждый
пиксель (точку
изображения) на
4 соседних



Картинка, снятая с
низким разрешением,
но увеличенная с
применением
интерполяции

Интерполяция (модуль `scipy.interpolate`)

 Пример:

```
from scipy.interpolate import interp1d
import numpy as np

x = [0, 1, 2]
y = [0, 1, 4]

f = interp1d(x, y, kind='quadratic')
print(f(1.5)) # Интерполированное значение
```

Создаём интерполяционную функцию `f`:

`kind='quadratic'` – значит, будет использоваться квадратичная интерполяция (парабола, проходящая через 3 точки). Тип сплайн-интерполяции может быть: `'linear'`, `'nearest'`, `'nearest-up'`, `'zero'`, `'slinear'`, `'quadratic'`, `'cubic'`, `'previous'`, `'next'`.

Теперь `f` – это функция, которую можно вызывать, как обычную, например: `f(1.5)`

Быстрое преобразование Фурье(модуль `scipy.fft`)



Возможности:

- Позволяет выполнять прямое и обратное БПФ (FFT), аналог `numpy.fft`, но быстрее.

Функция	Назначение
<code>fft</code>	Прямое БПФ
<code>ifft</code>	Обратное БПФ
<code>rfft</code>	Реальная БПФ
<code>fftfreq</code>	Частотная шкала

Быстрое преобразование Фурье(модуль `scipy.fft`)

- 📡 Что такое FFT?
- 📡 Быстрое преобразование Фурье (Fast Fourier Transform, FFT) — это метод разложения сигнала на сумму синусоид и косинусоид (компоненты частот).
- 📡 С помощью FFT можно:
 - 🕒 узнать, какие частоты присутствуют в сигнале,
 - 🕒 отфильтровать шумы,
 - 🕒 выполнить сжатие данных,
 - 🕒 анализировать временные ряды, звук, изображение и др.

Быстрое преобразование Фурье(модуль `scipy.fft`)

Создаём сигнал:

```
x = np.array([0, 1, 0, -1])
```

Это простой дискретный сигнал из 4 точек:

$x = [0, 1, 0, -1]$

Он представляет один период синусоиды.

Преобразование Фурье:

```
y = fft(x)  
y = array([0. +0.j, 0. -2.j, 0. +0.j, 0. +2.j])
```

Это означает, что в сигнале есть компонента с частотой, соответствующей индексу 1 и 3 (на частотах 1 и 3), амплитуды записаны в виде комплексных чисел (частота и фаза).

Быстрое преобразование Фурье(модуль `scipy.fft`)

Обратное преобразование Фурье:

```
ifft(y)
```

Функция `ifft` превращает сигнал обратно из частотной области во временную:

`x = [0,1,0,-1]`

Результат:

```
[ 0.+0.j  1.+0.j  0.+0.j -1.+0.j]
```

Быстрое преобразование Фурье(модуль `scipy.fft`)

 Где это используется?

- ⦿ Обработка аудио: выделение частот (низкие, средние, высокие)
- ⦿ Обработка изображений: фильтрация и сжатие
- ⦿ Анализ колебаний и вибраций
- ⦿ Радарные и медицинские сигналы (например, ЭКГ, МРТ)

Быстрое преобразование Фурье(модуль scipy.fft)

 Пример:

```
from scipy.fft import fft, ifft
import numpy as np

x = np.array([0, 1, 0, -1])
y = fft(x)
print(ifft(y))  # восстановление исходного сигнала
```

Преобразует сигнал x из временной области в частотную с помощью FFT.

Преобразует результат обратно во временную область с помощью обратного FFT (IFFT).

Показывает, что восстановленный сигнал совпадает с оригиналом.

Обработка сигналов(модуль scipy.signal)

Возможности:

- фильтрация (Butterworth, Chebyshev),
- свёртка и корреляция,
- спектрограмма и БПФ.



Обработка сигналов(модуль `scipy.signal`)

Пример:

Удалить шум из синусоидального сигнала, используя фильтр Баттерворта (Butterworth filter) и двунаправленную фильтрацию (`filtfilt`).

```
from scipy.signal import butter, firlfilt
import numpy as np
```

- `butter` — создает коэффициенты фильтра Баттерворта.
- `firlfilt` — применяет фильтр вперёд и назад, устраняя фазовые искажения.
- `numpy` — для работы с числовыми данными и генерации сигнала.

Обработка сигналов(модуль `scipy.signal`)

```
t = np.linspace(0, 1, 1000, endpoint=False)
```

Создаётся вектор времени `t`:

- от 0 до 1 секунды,
- 1000 точек (частота дискретизации = 1000 Гц),
- `endpoint=False` — не включает 1.0.

```
sig = np.sin(2*np.pi*5*t) + 0.5*np.random.randn(1000)
```

Создаётся сигнал:

- `np.sin(2*np.pi*5*t)` — чистая синусоида частотой 5 Гц;
- `0.5*np.random.randn(1000)` — случайный шум, амплитудой ± 0.5 .

То есть `sig` — зашумлённая синусоида:

$$\text{signal} = \sin(2\pi \cdot 5t) + \text{noise}$$

Обработка сигналов(модуль `scipy.signal`)

Создание фильтра Баттерворта:

```
b, a = butter(4, 0.1)
```

Здесь:

- 4 — порядок фильтра (чем выше, тем круче фильтрация);
- 0.1 — нормализованная частота среза (может быть задана промежутком):
 - от 0 до 1 (где 1 соответствует половине частоты дискретизации, т.е. 500 Гц),
 - значит, 0.1 соответствует 50 Гц ($0.1 * 500$).

Фильтр будет пропускать сигналы ниже 50 Гц, и подавлять более высокие частоты (в том числе шум).

`butter` возвращает:

`b`, `a` — коэффициенты числителя и знаменателя фильтра.

Обработка сигналов(модуль `scipy.signal`)

Применение фильтра:

```
filtered = filtfilt(b, a, sig)
```

filtfilt применяет фильтр дважды (вперёд и назад), чтобы:

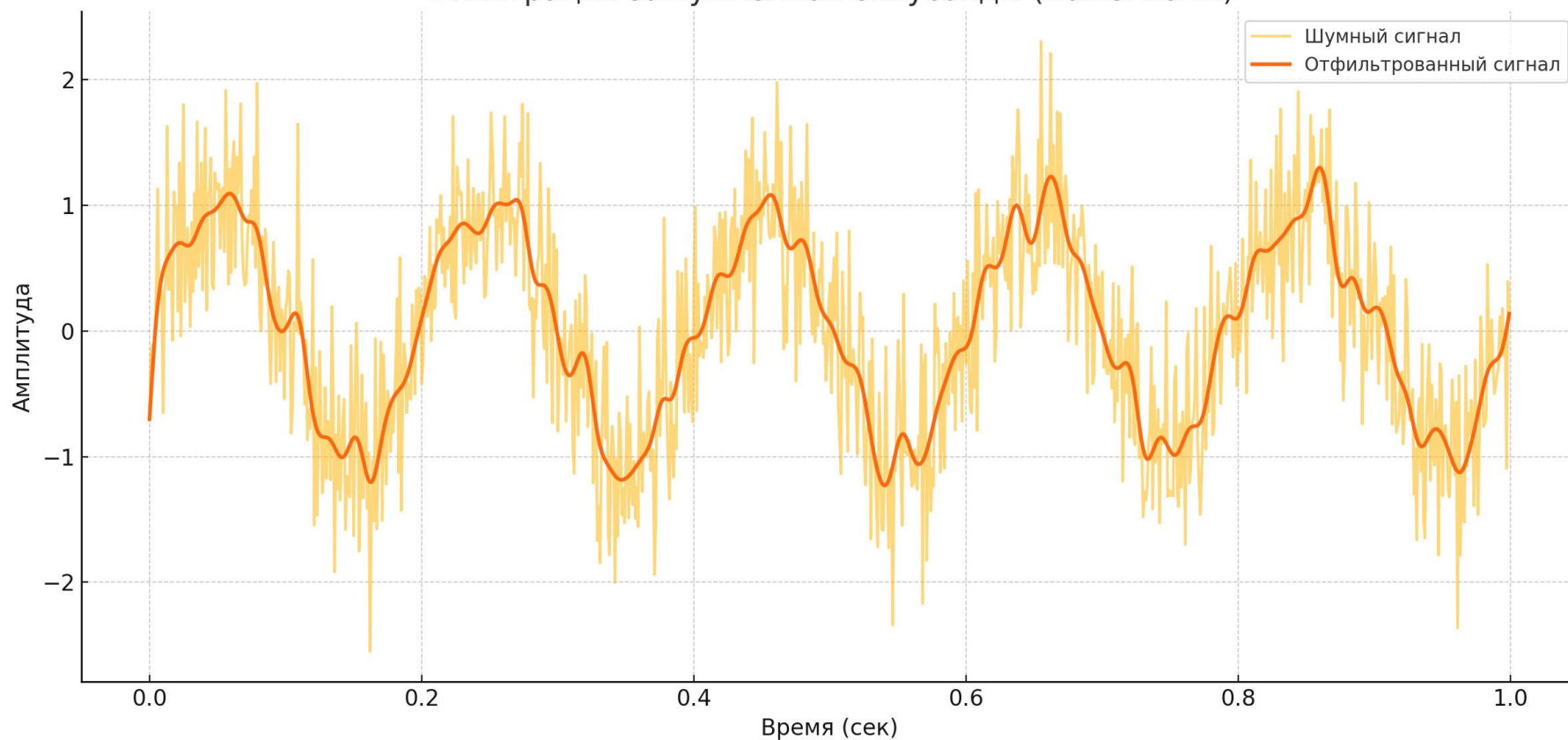
- устранить фазовые искажения,
- сохранить форму сигнала.

Результат — очищенный сигнал без высокочастотного шума.

Обработка сигналов (модуль `scipy.signal`)

Применение фильтра:

Фильтрация зашумлённой синусоиды (Butterworth)



Обработка сигналов (модуль `scipy.signal`)

🔊 Спектр сигнала через быстрое преобразование Фурье:



Статистика (модуль scipy.stats)

Возможности:

- генерация случайных выборок,
- проверка гипотез (t-тест, ANOVA),
- вычисление корреляций и распределений.



Статистика(модуль scipy.stats)

 Основные функции:

Функция	Назначение
<code>norm, binom</code>	Стат. распределения
<code>ttest_1samp</code>	t-тест для одной выборки
<code>pearsonr</code>	Коэффициент корреляции
<code>shapiro</code>	Проверка на нормальность

Статистика(модуль `scipy.stats`)

Пример:

Проверить гипотезу о том, что среднее значение выборки не отличается от заданного значения (в данном случае — от нуля).

Это называется одновыборочный t-тест (англ. one-sample t-test).

```
from scipy.stats import ttest_1samp  
import numpy as np
```

ttest_1samp — функция для выполнения t-теста для одной выборки;

numpy — для генерации случайных данных.

Статистика(модуль scipy.stats)

 Пример:

```
data = np.random.normal(loc=0, scale=1, size=100)
```

Генерируется случайная выборка data из 100 чисел, распределённых по нормальному закону:

loc=0 — математическое ожидание (среднее) = 0;

scale=1 — стандартное отклонение = 1;

size=100 — размер выборки = 100 элементов.

Результат:

```
[ 0.36, -0.52, 1.12, ..., -0.85 ]
```

Статистика(модуль scipy.stats)

 Пример:

```
print(ttest_1samp(data, popmean=0))
```

Проводится ****одновыборочный t-тест****, чтобы проверить ****гипотезу****:

Нулевая гипотеза (H_0):

Среднее значение выборки равно `popmean = 0`.

Альтернативная гипотеза (H_1):

Среднее значение ****отличается**** от нуля.

Что возвращает `ttest_1samp`?

```
TtestResult(statistic=np.float64(-0.16558436525306233),  
pvalue=np.float64(0.8688217213982216), df=np.int64(99))
```

Статистика(модуль scipy.stats)

 Пример:

```
TtestResult(statistic=np.float64(-0.16558436525306233),  
pvalue=np.float64(0.8688217213982216), df=np.int64(99))
```

statistic — t-статистика, показывает, насколько сильно среднее отклоняется от ожидаемого значения;

pvalue — p-значение, вероятность получить такие (или более экстремальные) данные, если H_0 верна.

statistic = -0.165 — t-значение небольшое → среднее не сильно отклоняется от 0;

pvalue = 0.86 — высокое значение (больше 0.05) → нет оснований отклонить гипотезу (если меньше 0,05 - есть статистически значимое отклонение от среднего 0);

df = 99 — число степеней свободы = 100 – 1. (Количество значений которые могут быть изменены независимо в выборке)

Статистика(модуль scipy.stats)



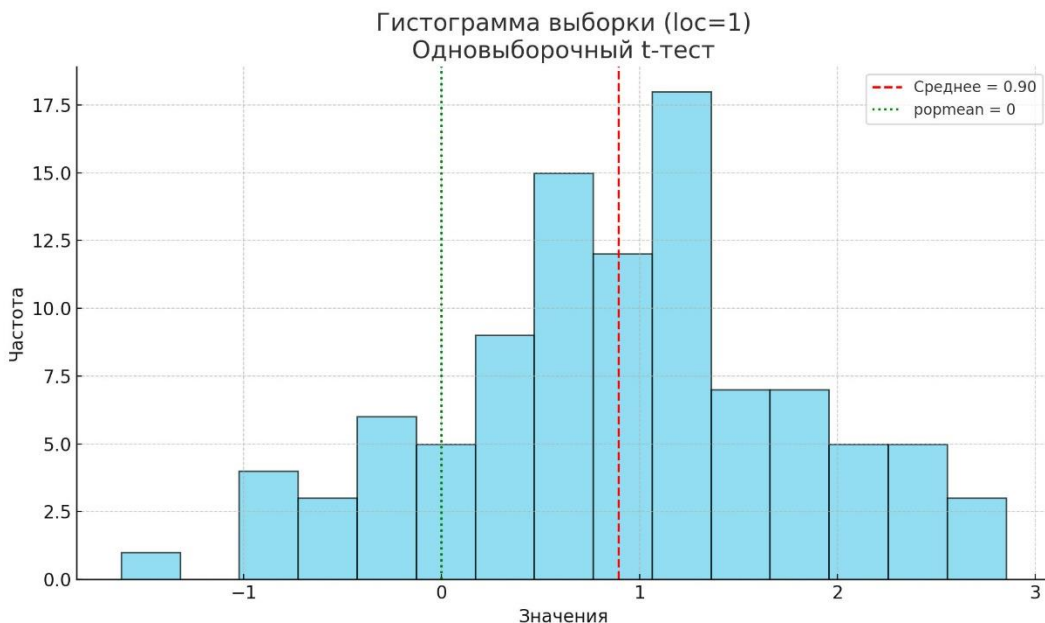
Пример:

```
data = np.random.normal(loc=1, scale=1, size=100)
```

Теперь $p_{\text{or mean}}=0$, но фактически данные сгенерированы со средним 1, и p-value будет маленьким → гипотеза отвергнется.

Статистика(модуль scipy.stats)

📊 Пример:



Голубые столбцы — гистограмма распределения случайной выборки.

Красная пунктирная линия — среднее значение выборки (в данном случае около 1).

Зелёная точечная линия — проверяемое значение среднего (0, портmean=0).

Статистика(модуль scipy.stats)



Где применяется t-тест?

- ⦿ Проверка эффективности новых лекарств (до/после лечения)
- ⦿ Сравнение среднего дохода группы с общенациональным
- ⦿ Анализ результатов экспериментов

Геометрия и расстояния (модуль `scipy.spatial`)



Используется для:

- поиска расстояний,
- построения KD-деревьев,
- триангуляции и выпуклой оболочки.

***Я ПОСЛЕ ОЧЕРЕДНОЙ ПЕРЕСТАНОВКИ
В ПЯТЁРОЧКЕ:***



Статистика(модуль scipy.stats)

Основные функции:

Функция / Класс	Назначение
<code>distance.euclidean(a, b)</code>	Евклидово расстояние между a и b
<code>distance_matrix(A, B)</code>	Матрица расстояний между наборами
<code>KDTree</code> / <code>cKDTree</code>	Быстрый поиск ближайших соседей
<code>ConvexHull(points)</code>	Построение выпуклой оболочки
<code>Delaunay(points)</code>	Триангуляция Делоне
<code>Voronoi(points)</code>	Диаграмма Вороного
<code>cdist(A, B)</code>	Все пары расстояний между двумя наборами
<code>pdist(X)</code>	Все попарные расстояния в одном наборе

Геометрия и расстояния(модуль `scipy.spatial`)

 Пример, поиск расстояния между двумя точками:

```
from scipy.spatial import distance
a = [1, 0]
b = [0, 1]
print(distance.euclidean(a, b))  #  $\sqrt{2}$ 
```

distance.euclidean – поиск в евклидовом пространстве (могут быть манхэттенское, косинусное и др.).

Геометрия и расстояния(модуль `scipy.spatial`)

🌀 Пример, найти ближайшую точку к заданной в наборе данных:

```
from scipy.spatial import KDTree
import numpy as np

points = np.random.rand(100, 2)
tree = KDTree(points)
dist, idx = tree.query([0.5, 0.5])

print(f"Ближайшая точка: {points[idx]}, расстояние: {dist}")
```

KDTree(points) – построение kd-дерева по точкам `points`.

tree.query([0.5, 0.5]) – поиск ближайшей точки в дереве к точке с координатами `[0.5, 0.5]`

Геометрия и расстояния (модуль `scipy.spatial`)

 Где применяется `scipy.spatial`?

 Вычисление расстояний между точками

- ⦿ Машинное обучение,
- ⦿ Анализ данных,
- ⦿ Картография (измерения между координатами).

 Работа с выпуклыми оболочками (Convex Hull)

- ⦿ Распознавание контуров объектов,
- ⦿ Построение минимальных ограничивающих областей,
- ⦿ 3D-моделирование.

 Поиск ближайших соседей

- ⦿ Рекомендательные системы,
- ⦿ Обработка изображений,
- ⦿ Классификация и кластеризация.

ИТОГИ

В данной лекции были рассмотрены следующие темы:

-  Обзор библиотеки SciPy
-  Основные модули библиотеки SciPy